

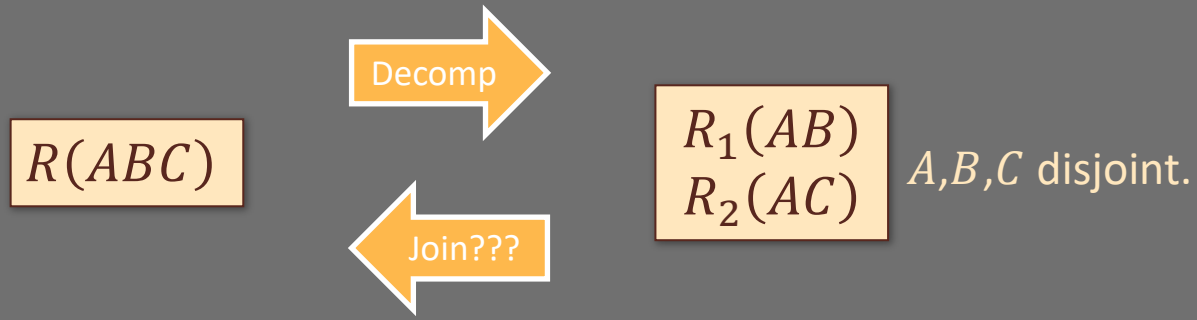
Decompositions in General

Desired properties

Don’t want to lose info during decomposition.

- Lossless-join
- Dependency preserving

Lossless-join decompositions



Decomposition breaks a table into pieces. Does joining restore the original table?
If so, decomposition is lossless.

When is decomposition lossless?

Defn: … iff $AB \bowtie AC = ABC$.

Theorem: … iff $A \to B$ or $A \to C$.

Intuition: 1-many relationship

Lossless-join example

name	price	category
Gizmo	19.99	Gadget
OneClick	24.99	Camera
Gizmo	19.99	Camera

$\{name\} \rightarrow \{price\}$



name	price
Gizmo	19.99
OneClick	24.99
~~Gizmo~~	~~19.99~~

name	category
Gizmo	Gadget
OneClick	Camera
Gizmo	Camera

A = {name}
B = {price}
C = {category}

Lossy-join example

name	price	category
Gizmo	19.99	Gadget
OneClick	24.99	Camera
Gizmo	19.99	Camera

$\{name\} \rightarrow \{price\}$

name	price	category
Gizmo	19.99	Gadget
OneClick	24.99	Camera
Gizmo	24.99	Camera
OneClick	19.99	Camera
Gizmo	19.99	Camera





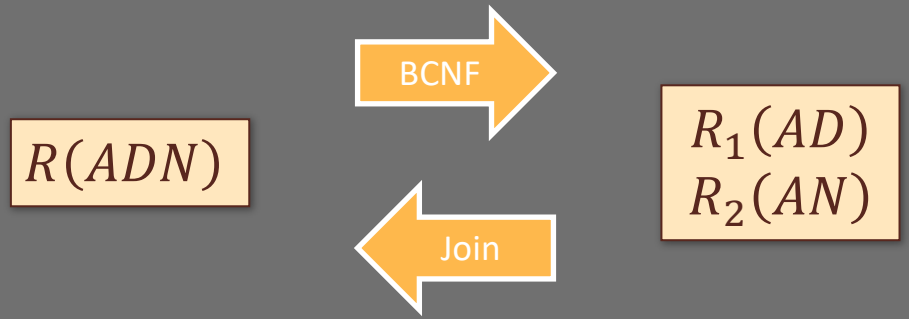
Loses the association between name and price.

price	category
19.99	Gadget
24.99	Camera
19.99	Camera

name	category
Gizmo	Gadget
OneClick	Camera
Gizmo	Camera

A = {category}
B = {price}
C = {name}

BCNF decomposition is lossless-join



BCNF decomposition guarantees $A \rightarrow D$.

However, BCNF decomposition can lose FD information

R(name, company, category)
{category, company} → {name}
{name} → {company}

Decomp

R_1(<u>name</u>, <u>category</u>)
R_2(<u>name</u>, company)
{category, company} → {name}
{name} → {company}

"Bad" FD

Can't enforce "nonlocal" FD.

Keys: {category, company}, {category, name}

name	company	category
Gizmo	GizmoCorp	Gadget
GizmoPlus	GizmoWorks	Gadget

Join

name	category
Gizmo	Gadget
GizmoPlus	Gadget

name	company
Gizmo	GizmoCorp
GizmoPlus	GizmoWorks



What can we do about this?

- Accept this trade-off.
- Enforce those FDs after joining!
  - Expensive.
- Weaken decomposition so that FDs not lost.
  - E.g., 3NF

3NF

BCNF vs. 3NF

BCNF

Relation R is in BCNF

iff

for every FD $A \rightarrow B$ in R,

- $A \rightarrow B$ is trivial, or
- A is a superkey for R.

3NF

Relation R is in 3NF

iff

for every FD $A \rightarrow B$ in R,

- $A \rightarrow B$ is trivial, or
- A is a superkey for R, or
- every element of B is part of some key.

3NF decomposition is lossless-join

Doesn’t decompose any relation that BCNF doesn’t.

Thus has same property that

if ABC decomposes to AB and AC,

then $A \rightarrow B$ or $A \rightarrow C$.

3NF doesn't lose FD information

R(name, company, category)
{category, company} → {name}
{name} → {company}

BCNF: "Bad"
3NF: "Good" because **company** part of some key.

Keys: {category, company}, {category, name}

3NF allows some redundancies/anomalies

name	company	category
Gizmo	GizmoWorks	Gadget
Gizmo	GizmoWorks	Camera
GizmoPlus	GizmoWorks	Gadget
GizmoPlus	GizmoWorks	Camera
NewThing	NULL	Gadget

Redundancy. Repeating product's company.

Insertion anomaly. Product not yet made by any company.

BCNF vs. 3NF

BCNF

- Lossless-join
- Can lose FD info

3NF

- Lossless-join
- Doesn’t lose FD info
- Allows more redundancy
- Decomposition algorithm more complicated

Intuition for BCNF & 3NF

(3NF) “[Every] non-key field must provide a fact about the key, the whole key, and nothing but the key.”

– Bill Kent

(BCNF) “Each attribute must represent a fact about the key, the whole key, and nothing but the key.”

– Chris Date